

Quadrotor path planning using A* search algorithm and minimum snap trajectory generation

Youkyung Hong  | Suseong Kim  | Yookyung Kim | Jihun Cha

Autonomous Unmanned Vehicle
Research Department, Electronics and
Telecommunications Research Institute,
Daejeon, Republic of Korea

Correspondence

Suseong Kim, Autonomous Unmanned
Vehicle Research Department, Electronics
and Telecommunications Research
Institute, Daejeon, Republic of Korea.
Email: suseongkim@etri.re.kr

Funding information

ETRI grant, Grant/Award Numbers:
2017-0-00067, Development of ICT Core
Technologies

Abstract

In this study, we propose a practical path planning method that combines the A* search algorithm and minimum snap trajectory generation. The A* search algorithm determines a set of waypoints to avoid collisions with surrounding obstacles from a starting to a destination point. Only essential waypoints (waypoints necessary to generate smooth trajectories) are extracted from the waypoints determined by the A* search algorithm, and an appropriate time between two adjacent waypoints is allocated. The waypoints so determined are connected by a smooth minimum snap trajectory, a dynamically executable trajectory for the quadrotor. If the generated trajectory is invalid, we methodically determine when intermediate waypoints are needed and how to insert the points to modify the trajectory. We verified the performance of the proposed method by various simulation experiments and a real-world experiment in a forested outdoor environment.

KEYWORDS

A* search algorithm, minimum snap trajectory, path planning, quadrotor

1 | INTRODUCTION

The use of multirotor unmanned aerial vehicles (commonly known as quadrotors or drones) is rapidly increasing in numerous public and industrial sectors for applications, such as inspection, surveillance, and reconnaissance. For quadrotors to fly as fully autonomous systems, various technologies are required, such as positioning by navigation, state recognition by fusing multisensor information, and environmental perception. Above all, we focus on the quadrotor path planning problem in this study, which is essential for safe flight by avoiding collisions with obstacles. In the field of quadrotor path planning, several attempts have been made to achieve real-time requirements (that is, real-time path generation by reflecting the changes in the surrounding environment

and the state of a quadrotor) [1] and accounting for dynamic feasibility (that is, the suitability of the generated path a quadrotor follows) [2]. In addition, the performance of path planning has been verified through real-world experiments, and the level of path planning has been judged by how much the complex environment can be considered [3] as well as how fast the quadrotor can fly [4]. Quadrotor path planning is an open challenge, and no consensus on the best strategy has emerged.

Generally, the path planning problem consists of global and local path planning [5]. Global path planning entails trajectories that only satisfy geometric constraints to avoid collisions with surrounding obstacles. The output of global path planning is a set of waypoints, and the dynamics of quadrotors are not considered in global path planning. To date, the methods proposed for global path

planning can be divided into the search- and sampling-based algorithms [2]. Search-based algorithms convert obstacle information on the map into graphs for use in path searches [6,7]. The most commonly employed search-based algorithm is the A* search algorithm, which defines a cost function, such as traveled distance, for each point and searches the path with the minimum cumulative cost from a starting to a destination point. Conversely, sampling-based algorithms have been proposed to find the appropriate path efficiently, even in complex environments. Among the sampling-based algorithms, the most commonly employed is the rapidly exploring random tree (RRT) algorithm, which gradually builds a graph from a starting to a destination point using random samples [8,9].

Local path planning considers the quadrotor's dynamic characteristics and creates a smooth and appropriate trajectory for it to follow [10]. Waypoints determined by global path planning can simply be connected using straight lines. However, control commands generated to follow this trajectory may produce actuator commands that exceed the actuator's actual limits because the trajectory of the straight line has infinite curvature at the waypoints. Therefore, various methods have been proposed for quadrotor path planning that allows quadrotors to follow efficiently and accurately by accounting for the quadrotor dynamics [11,12]. The minimum snap trajectory generation (MSTG), proposed by Mellinger and Kumar [13], is a representative method for producing a dynamically executable quadrotor trajectory. In addition, Richter et al. [14] proposed an analytical method to generate minimum snap trajectory efficiently by reformulating a constrained quadratic programming (QP) as an unconstrained QP.

This study aims to develop a practical path planning method that combines the A* search algorithm (for global path planning) and MSTG (for local path planning). Although the A* search algorithm and MSTG have been proven in their respective fields [15–17], to the best of our knowledge, no previous research has combined them and validated the resulting performance. Further, this study aims to prevent the disadvantages inherent in the conventional quadrotor path planning that separately designs global and local path planning. The disadvantage of conventional quadrotor path planning is that new collisions with surrounding obstacles can occur in the process of smoothly connecting two adjacent waypoints in local path planning because the connection between waypoints is a straight line, unlike the smooth trajectory in global path planning. Richter et al. [14] reported that after local path planning, a particular trajectory segment might intersect obstacles, but adding an intermediate waypoint between two waypoints solves this problem.

However, some later studies have argued that this alternative method does not always guarantee a collision-free trajectory [18]. Few studies have systematically investigated—(i) where to add intermediate waypoints, (ii) when the points are required, and (iii) how the points are inserted.

In this study, the following efforts are made to create the best combination of the A* search algorithm and MSTG. Instead of using all waypoints determined by the A* search algorithm, we consider extracting only the essential waypoints—waypoints necessary to generate smooth trajectories. An appropriate time between two selected adjacent waypoints is allocated, and then smooth trajectories connecting the extracted waypoints are generated. If the previously generated trajectory is invalid, we methodically determine when and how to insert an intermediate waypoint to modify the current trajectory. In addition, various simulations are performed to verify the feasibility and performance of the proposed path planning. The success rate is quantitatively analyzed depending on the obstacle density and quadrotor speed. Further, we construct a fully autonomous quadrotor testbed, including an environmental perception module to detect the surrounding obstacles and a flight control module to ensure a quadrotor follows the given trajectory accurately. The performance of the quadrotor testbed is verified by real-world outdoor experiments.

The main contributions of this study are summarized as follows. First, combining the A* search algorithm and MSTG is proposed to generate a collision-free and dynamically feasible trajectory in real time. Second, the most reliable performance validation is performed from massive simulations to the challenging outdoor experiment with the complete quadrotor testbed. To the best of our knowledge, this is the first dynamically feasible and real-time path planning method capable of flying at the fastest speeds in a forested outdoor environment.

The rest of this study is organized as follows. First, Section 2 first provides a literature review of quadrotor path planning. Section 3 details the proposed path planning method that combines the A* search algorithm and the MSTG. Section 4 reports on the performance of various simulation experiments to validate the proposed path planning method. Section 5 describes the results of the outdoor experiment conducted in a dense forest with uneven terrain. Section 6 discusses the conclusions of this study and possible directions for future research.

2 | RELATED WORKS

A considerable number of studies have been conducted on quadrotor path planning. However, for brevity, only

recent works most relevant to this study are discussed in this section. As mentioned above, this study focuses on quadrotor path planning consisting of global and local path planning. There is a lot of variant path planning, depending on which method is adopted for global or local path planning. Related works are reviewed in the following order—studies that adopted different global path planning; studies that adopted different local path planning; and studies that used different methods for both global and local path planning.

First, previous works that adopted other global path planning that is different from the A* search algorithm are as follows. Richter et al. [14] used the RRT algorithm for global path planning. The RRT algorithm is generally known to be computationally complex and requires additional effort to ensure real-time performance [19]. Liu et al. [5,11] generated a set of motion primitives, which are polynomial trajectories from one state to another within a specified duration. Then, they searched for the optimal sequence of primitives for global path planning. However, the refined trajectory might be unsafe in the local path planning process that produces a smooth trajectory (a typical disadvantage that occurs when using a combination of global and local path planning). Besides, the proposed path planning was verified through an indoor experiment where a motion capture system existed; only a few panels were installed.

Second, previous studies that adopted local path planning that is different from the MSTG are as follows. Ding et al. [20] designed the smooth trajectory based on B-spline curves instead of polynomial trajectories. However, the disadvantage of the B-spline is that the control points cannot be set intuitively because the generated curve does not accurately penetrate the control points. Liu et al. [2] provided important insight for solving the fundamental problems inherent in quadrotor path planning, consisting of global and local path planning. In their works, the smooth trajectory was optimized by determining the flight corridor without collisions and adding constraints that the trajectory must be located inside the corridor. An outdoor experiment was conducted in an environment similar to that in this study. However, it is unfortunate that the resulting trajectory does not pass through the forest and bypasses a less complicated area.

Finally, previous studies that adopted both global and local path planning that is different from this study are as follows. Gao et al. [21] used the informed sampling scheme for global path planning where safe balls connecting from a starting to a destination point are randomly generated, and then an initial straight line safe trajectory is searched on the graph. In addition, heuristic optimization was proposed for local path planning. However, heuristic optimization has an inherent limitation

that if a solution does not converge within a given time, a local optimal solution is determined instead of a global optimal solution. Zhou et al. [12] proposed combining the kinodynamic path searching and non-uniform B-spline curve optimization for quadrotor path planning. However, they mentioned that aggressive flight experiments were conducted in a cluttered environment, and the maximum flight speed was only 2.5 m/s.

In summary, several attempts have been made for quadrotor path planning consisting of global and local path planning. There are many variations of quadrotor path planning, depending on which algorithm is adopted for global or local path planning. However, few previous studies have conducted outdoor experiments similar to this study, and none have achieved flight speed comparable to this study.

3 | ALGORITHM DESCRIPTION

The overall procedure of the proposed path planning method is illustrated in Figure 1. The detailed description for each part is as follows.

3.1 | Occupancy Grid Map

Environmental perception should be conducted before the proposed path planning method is executed. This requires that environmental information indicating whether a point in the world coordinate is occupied (that is, inside an obstacle) or free (that is, outside an obstacle) should be acquired. Areas not detected by sensors are designated as free, and therefore all points are initially considered free. Immediately, the sensors are activated; the environmental information within the detection range (that is, within a certain radius of a quadrotor) can be acquired. As the quadrotor moves, the detected area is expanded, the environmental information is updated, and the path planning is updated in real time.

Once the environmental information is obtained, a binary occupancy grid map (OGM) can be produced. The OGM can be projected as a large fixed sized cube such that $[x, y, z]^T \in \{x_{\min} \leq x \leq x_{\max}, y_{\min} \leq y \leq y_{\max}, \text{ and } z_{\min} \leq z \leq z_{\max}\}$. The map is divided by a certain resolution interval to represent the points previously expressed by $[x, y, z]^T$ as indexed cubes of the form $[i, j, k]^T$. These indexed cubes have a value of either zero (that is, free) or one (that is, occupied) depending on the environmental information. To account for the dimension of the quadrotor, adjacent cubes (see gray squares in Figure 1A) around the occupied cube (see black squares in Figure 1A) are also considered occupied by inflating

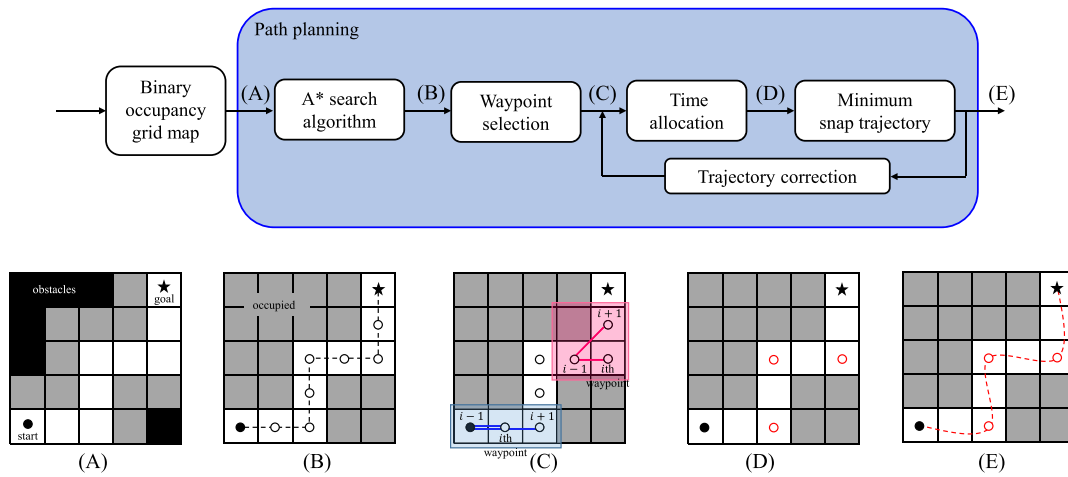


FIGURE 1 The framework of the proposed path planning

the obstacles. The inflated map ensures a safe distance between the quadrotor and the surrounding obstacles; therefore, the quadrotor avoids the obstacles.

In this study, the point cloud library (PCL) [22] is adopted to acquire the environmental information. Note that any method can be used for environmental perception provided the environmental information, indicating whether a point is occupied or not, can be obtained.

3.2 | A* search algorithm and waypoint selection

When the OGM is prepared, we define the starting and destination cubes inside the map. The A* search algorithm—a typical graph search algorithm—searches a path with the smallest cost (generally the shortest distance) from the starting to the destination cube. For implementing the A* search algorithm, it is assumed that the quadrotor only moves to six adjacent cubes—up, down, front, back, left, and right—from the current cube. In other words, movement in the diagonal direction is not considered in this study. Therefore, the cost required to move to the adjacent cubes can be considered the same. Because the pseudocode for the A* search algorithm is commonly available in the literature, we do not describe the A* search algorithm in detail.

The result of the A* search algorithm is a large set of cubes (called waypoints) that connect the starting and destination cubes on a finely divided map. As an example, shown in Figure 1B, if a $5\text{ m} \times 5\text{ m}$ two-dimensional (2D) map is divided by a $1\text{ m} \times 1\text{ m}$ grid, then the A* search algorithm generates seven waypoints, excluding the starting and destination points. It is questionable whether it is necessary to generate a smooth trajectory through all

points determined by the A* search algorithm as shown in Figure 1B. Therefore, among dozens or hundreds of waypoints obtained from the A* search algorithm, this study extracts certain waypoints, which are essential to avoid surrounding obstacles. The waypoint selection mechanism is motivated by GitHub repository [23] and briefly illustrated in Figure 1C. As shown in Figure 1C, to determine whether the i th waypoint obtained by the A* search algorithm should be considered for smooth trajectory generation or not, two straight lines are required: one connecting the $i - 1$ th and i th waypoints and the other connecting the $i - 1$ th and $i + 1$ th waypoints. As the blue box in Figure 1C shows, if the two lines do not overlap with obstacles, then the i th waypoint is not considered to produce a smooth trajectory. Conversely, as illustrated in the red box of Figure 1C, if the line connecting the $i - 1$ th and $i + 1$ th waypoints overlaps with obstacles, then the i th waypoint generates a smooth trajectory. In this manner, we can select the essential waypoints, as shown in Figure 1D.

3.3 | Time allocation

It is assumed that two adjacent waypoints constitute one segment. For example, in Figure 1D, there are four segments from the starting point to the destination point. Once all segments from the starting to the destination point are determined, the amount of time that should be allocated to each segment must be considered. The simplest solution is to allocate the same amount of time for all segments. However, this simple method may allow the quadrotor to accelerate in long segments or decelerate in short segments. Therefore, it is necessary to allocate the appropriate time for each segment. In this study, the

following process is applied. The total flight time is initially calculated by dividing the straight line distance from the starting to the destination point by the predetermined constant speed of the quadrotor. Then, the approximated total flight time is distributed proportionally to the square root of the straight line distance for each segment. This technical approach is the most straightforward method for time allocation in each segment, and the state-of-the-art method for optimal time allocation is described in Richter et al. [14].

3.4 | Minimum snap trajectory generation

Generally, the dynamics of the quadrotor are nonlinear and underactuated. The state vector $\mathbf{x} \in \mathbb{R}^{12}$ to describe the dynamics of quadrotor can be represented as follows:

$$\mathbf{x} = [x, y, z, u, v, w, \phi, \theta, \psi, p, q, r]^T, \quad (1)$$

where (x, y, z) and (u, v, w) denote position and velocity in the global coordinate frame, respectively, and (ϕ, θ, ψ) and (p, q, r) denote Euler angle and angular velocity in the body-fixed coordinate frame, respectively. In addition, the control input vector $\mathbf{u} \in \mathbb{R}^4$ is given by the force u_1 from all the propellers and roll, pitch, and yaw moments (u_2, u_3, u_4) as follows.

$$\mathbf{u} = [u_1, u_2, u_3, u_4]^T. \quad (2)$$

However, Mellinger and Kumar [13] showed that the dynamics of the quadrotor are differentially flat when position (x, y, z) and yaw angle ψ are considered as the flat output vector $\sigma \in \mathbb{R}^4$.

$$\sigma = [x, y, z, \psi]^T. \quad (3)$$

In other words, by defining the position and yaw angle as the flat output vector, the state and control input vectors of the quadrotor are expressed as algebraic functions of the flat output vector, and its derivatives as follows:

$$(\mathbf{x}, \mathbf{u}) = f(\sigma, \dot{\sigma}, \ddot{\sigma}, \ddot{\ddot{\sigma}}). \quad (4)$$

Therefore, the path planning problem for the quadrotor becomes a position and yaw angle design problem owing to differential flatness.

If it is assumed that the entire path from the starting to the destination point comprises M segments, then there are $M - 1$ waypoints, excluding the starting and destination points. Suppose that for each segment, the position and yaw angle are represented by an independent polynomial of order N for time t as follows:

$$\sigma_i(t) = p_{0,i} + p_{1,i}t + \dots + p_{N,i}t^N \text{ for } i = 1, \dots, 4, \quad (5)$$

with the polynomial coefficients $\mathbf{p} = [p_{0,i}, p_{1,i}, \dots, p_{N,i}]^T \in \mathbb{R}^{4(N+1)}$. Then, the path planning problem becomes a problem of finding the coefficient $\mathbf{p}^*$ that minimizes the integral of the square of the fourth derivative of position (that is, snap) and the square of the second derivative of the yaw angle. The optimization problem can be formulated as constrained QP by enforcing initial, final, and intermediate conditions (that is, continuity of the first four derivatives of position and the first two derivatives of yaw angle at each waypoint). The detailed procedure to formulate and solve the constrained QP can be found in Mellinger and Kumar[13]. The analytic method for reformulating the constrained QP as the unconstrained QP is described in Richter et al. [14].

3.5 | Trajectory correction

As mentioned previously, the trajectory correction is essential in conventional quadrotor path planning consisting of global and local path planning because new collisions with surrounding obstacles can occur in local path planning. In this study, trajectory correction is performed by determining better intermediate waypoints to modify the existing trajectory when the existing trajectory is invalid. As shown in Figure 2, we systematically defined where intermediate waypoints were needed and included the process of adding intermediate waypoints to the existing framework, which minimized the conventional method's disadvantages. If, in the segment where the quadrotor is positioned (that is, the first segment in Figure 2A), the maximum deflection distance from the straight-line distance is 1 m or more (that is, the first case shown in Figure 2B) or a smooth curve intersects an obstacle (that is, the second case shown in Figure 2C), then this study determined that an intermediate waypoint is required. The position of the intermediate waypoint is the midpoint of two points that make up the currently located segment (that is, the position of the quadrotor and first waypoint), as shown in Figure 2D.

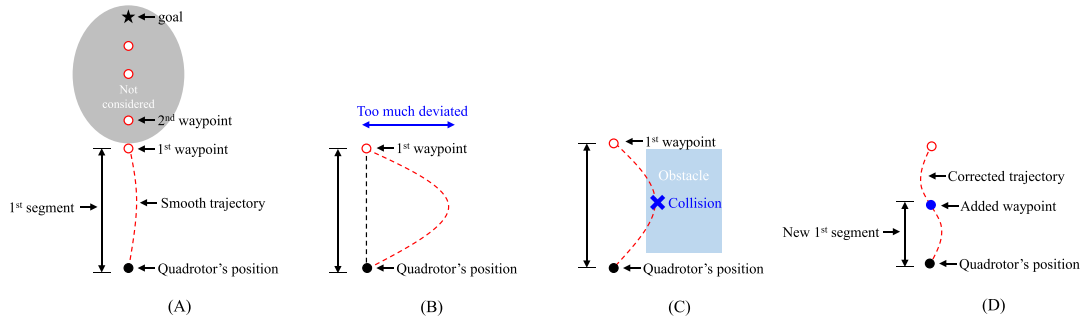


FIGURE 2 Illustration of trajectory correction: (A) before trajectory correction, (B) the first case that requires trajectory correction, (C) the second case that requires trajectory correction, and (D) after trajectory correction

4 | SIMULATION EXPERIMENTS

In this section, the performance of the proposed path planning is assessed in two simulation experiments. In the first part, the feasibility and the computation time of the proposed path planning method are evaluated at each iteration step. In the second part, Monte Carlo simulations were performed to prove that the proposed method guarantees collision-free flight in various environments and on multiple runs.

4.1 | Design of simulation experiments

We used PX4 Software In The Loop (SITL) configured with the Gazebo simulator to simulate the physical model and the flight controller of a quadrotor [24]. By using the simulator, we mimicked the quadrotor dynamics accurately because it considers not only rigid body dynamics but also complex aerodynamic effects such as rotor drags [25]. The SITL was also configured to run PX4 firmware (v1.9.0) as the flight control software used in the actual flight controller. Therefore, it is possible to conduct realistic simulations by considering actual quadrotor dynamics and its controller. The simulator was performed on a laptop PC with a 2.6-GHz Intel i7 CPU and 16-GB RAM running the Ubuntu operating system.

In the simulation environment, we created a finite configuration space, where the quadrotor could fly in the range of 0 m to 30 m in the x -axis, -15 m to 15 m in the y -axis, and 0 m to 5 m in the z -axis. To perform obstacle avoidance with the proposed method, we randomly placed cylinder obstacles having a height of 5 m and a radius of 1 m, as shown in Figure 3. The obstacle density in 2D space was 0.10 objects/m², which means the ratio of the occupied to the free space area.

To confirm the proposed path planning method, we customized a quadrotor based on a 3DR IRIS drone, which was equipped with a three-dimensional

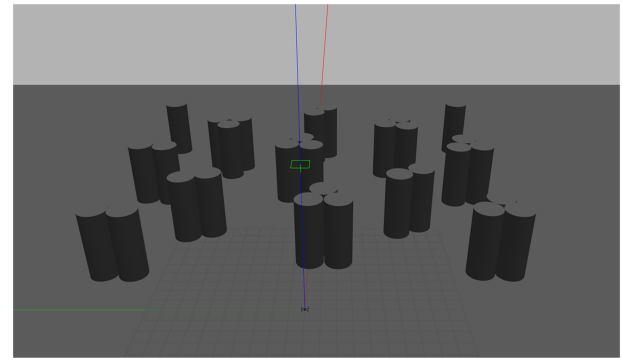


FIGURE 3 Simulation environment in Gazebo

(3D) LiDAR (Velodyne, VLP-16). We configured the LiDAR to generate 57 600 sample points at 5 Hz, and the field of view was 360° and 16°, respectively, in the horizontal and vertical directions. The detection range of the LiDAR was set to 1 m to 15 m. The point clouds from the LiDAR were used to build the OGM of the configuration space through the PCL. As the new point clouds became available, we immediately updated the environmental information to establish whether each grid was occupied or not. Using the latest OGM, we executed the proposed path planning method to find the obstacle-free trajectory. Regarding the parameters of path planning, we used an eighth-order polynomial to generate the minimum snap trajectory, and the desired speed was set to 2 m/s, as mentioned in Section 3.3. We generated a new trajectory at 5 Hz because the LiDAR reports new information at the same rate. The computed minimum snap trajectory was the polynomial wherein the variable is time. Hence, once a new trajectory was generated, we evaluated the desired position, velocity, yaw, and yaw rate from the computed polynomial at the current time and sent those values to the flight controller in the simulator. We employed MAVROS [26] as the interface between our trajectory planner and the flight controller in the simulator. The computed desired values were transmitted to the flight controller at 50 Hz.

For a safer operation, the emergency stop was used in the following cases [2]. First, we stopped the quadrotor when the difference between the desired and actual trajectories exceeded the specified level set to 2 m in our experiments. The second case was that no trajectory was generated within 5 Hz. That is, when the time required to compute the entire trajectory exceeded 0.2 s, the quadrotor switched to hovering mode. Lastly, the emergency stop was activated when the A* search algorithm failed to search the global path connecting the starting to the destination point. This occurs when the starting or destination point is considered occupied, or when the sensor noise blocks all paths.

4.2 | Performance evaluation

The first simulation experiment evaluates the feasibility and the computational efficiency of the proposed path planning method. Figure 4 shows the trajectory that updated over time for a case where the coordinates of the starting and destination points were set to (0, 0, 2) and (30, 0, 3), respectively. Note that the starting and destination points are always free. After taking off from the ground (0, 0, 0) in manual flight mode, the quadrotor platform was switched to the off-board mode when the altitude reached 2 m and then moved to the destination point. Note that the off-board mode enables the companion computer to control the quadrotor. The trajectories were updated 74 times during the flight, and of those, 25 are illustrated in Figure 4. As shown in Figure 4, the straight path was initially created because no obstacles were recognized immediately after

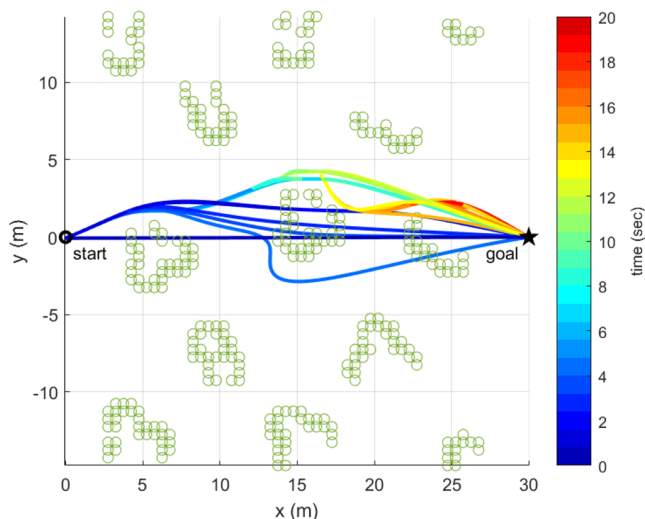


FIGURE 4 The proposed path planning over time

the simulation started (that is, when the time was zero). As the sensor initialized and recognized the surrounding obstacles, the proposed path planning method generated a feasible path that bypassed the detected obstacles.

Table 1 summarizes the average, minimum, and maximum run time generating trajectory in the simulation. Therefore, it is safe to claim that the proposed path planning method has low enough computational complexity, sufficient to run in real time (5 Hz) in flight.

4.3 | Monte Carlo simulation

The second simulation experiment examined whether the proposed path planning method guarantees collision-free flight in various environments and multiple runs. Monte Carlo simulations were conducted 1000 times, where the initial position was randomly assigned. The ranges of the start x , y , and z point were uniformly distributed from 0 m to 0.5 m, from -12 m to 12 m, and from 1.5 m to 3 m, respectively. The destination point was fixed to be the same point in Section 4.2.

Figure 5 shows the Monte Carlo simulation results. In Figure 5, the solid lines denote the actual trajectories traveled by the quadrotor. Except in 103 of 1000 simulations, the quadrotor successfully carried out an automatic flight from the starting to the destination point without the emergency stop mentioned in Section 3.1. In other words, the success rate was 0.897. Most emergency stops happened when the difference between the desired and actual trajectories was greater than 2 m.

TABLE 1 Trajectory generation run time

	Average	min	max
Time (s)	0.0392	0.0160	0.0840

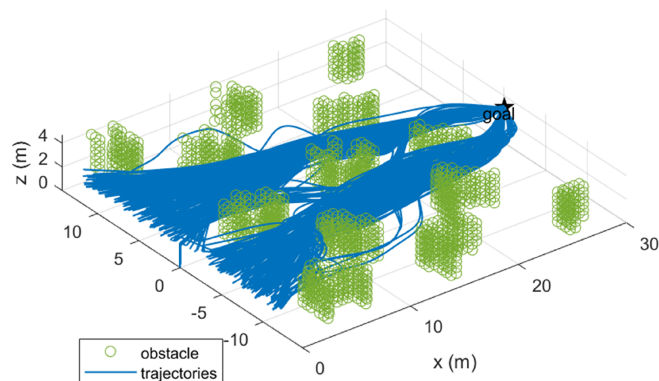


FIGURE 5 Trajectories followed in 1000 simulations

These results were also related to the performance of the flight controllers. In future research, we would consider developing better flight controllers and incorporating them into existing frameworks to further improve success rates.

Additional Monte Carlo simulations were performed for various obstacle densities and flight speeds to compare the results with previous works in the literature [19]. The obstacle density varied from 0.05 to 0.30. For example, the environment with an obstacle density of 0.3 contained 86 circular cylinder obstacles. The desired flight speed was set to 1 m/s and 2 m/s. The size of the map, type of obstacle, and setting of the starting and destination points were the same as the previous 1000 simulations of this study. The overall results are summarized in Table 2, and there were 100 trials for each obstacle density. As a result, when the average flight speed was 1 m/s, the success rate of the proposed method was higher than in Oleynikova et al. [19], where the success rates were less than 0.8, 0.6, and 0.5 respectively at 0.20, 0.25, and 0.30 obstacle densities.

5 | REAL-WORLD EXPERIMENT

In this section, to evaluate the proposed path planning method in a real-world scenario, we performed an outdoor experiment in a dense forest with a variable ground

TABLE 2 Success rates depending on the various obstacle densities and desired flight speeds

		Obstacle density (objects/m ²)					
		0.05	0.10	0.15	0.20	0.25	0.30
Speed	1	0.99	0.94	0.91	0.87	0.82	0.65
(m/s)	2	0.99	0.90	0.85	0.73	0.53	0.40



FIGURE 6 Quadrotor hardware platform

height. Similarly, as in the simulation experiments performed in Section 4, we built a custom quadrotor with the 3D LiDAR (Velodyne, VLP-16) that was installed on top of the quadrotor. We used a Pixhawk4 that runs PX4 firmware (v1.9.0) as a flight control computer (FCC). The FCC's role was to estimate the states, such as position, velocity, and attitude, as well as compute motor inputs to control the motion of the quadrotor. In addition, the quadrotor was equipped with an NVIDIA TX2 as a companion onboard computer. The onboard computer was connected to the LiDAR and FCC to compute the OGM from the point clouds, perform path planning, and transmit the desired position, velocity, yaw, and yaw rate to the FCC. Hence, all the presented approaches demonstrated the value of using one small-sized onboard computer rather than a ground workstation, which would not be practical in many situations. Once the desired states were transmitted to the FCC, it generated control inputs from the quadrotor to follow the desired trajectory. Including all the onboard peripheral devices, the quadrotor hardware platform's tip-to-tip length is 0.73 m, and its weight is 2.2 kg, as shown in Figure 6. All the hardware details are listed in Table 3.

Figure 7 shows the outdoor experiment scenario where the quadrotor has no prior knowledge of the environment; it only knows of the destination point in the world coordinate. The starting point was intentionally set for the quadrotor to traverse the forest without flying around it. As in the simulation experiments, after taking off in manual flight mode, the quadrotor was switched to the off-board mode when the altitude was above 1 m and then moved to the destination point.

Dozens of experiments have been conducted in the same environment, but only one of the most representative results is illustrated in Figures 8 and 9. Figure 8

TABLE 3 Hardware specification

Quadrotor platform	Frame	Artcopter T-18
	Motor	T-motor MN5212
	ESC	T-motor Alpha 40 A
	Propeller	T-motor MF 1604
	Battery	Tattu plus 16 000 mAh
Sensors	GPS	ublox M9P
	3D LiDAR	Velodyne Puck lite VLP-16
Computers	Flight controller	Pixhawk4 (PX4)
	Mission computer	NVIDIA TX2
	Carrier board	Auvideo J120



FIGURE 7 Satellite photo (left) and field photo (right) of the outdoor experiment scenario

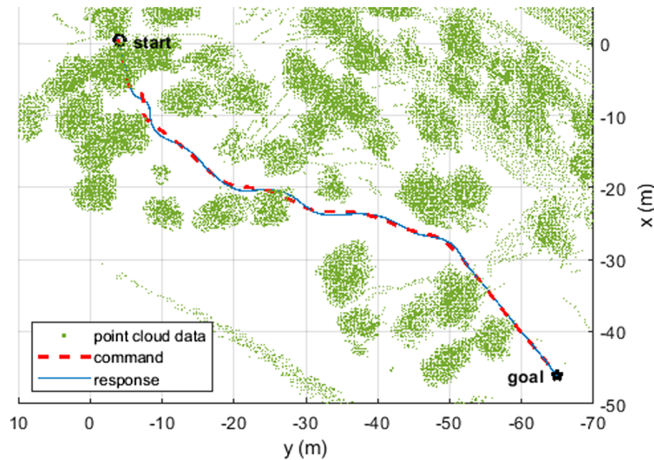


FIGURE 8 Environmental perception results through point cloud library (PCL) and trajectories

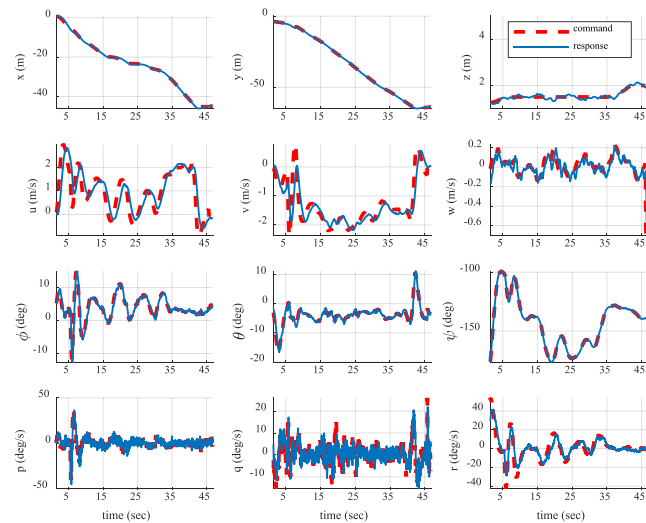


FIGURE 9 Time histories of states

shows the trajectory generated by the proposed path planning method and the actual trajectory the quadrotor traveled. Figure 9 shows the time histories of the states,

including positions, velocities, Euler angles, and angular rates, recorded in the outdoor experiment. The total distance traveled by the quadrotor was 84.23 m, and the quadrotor traveled at speeds of up to 3.22 m/s. Note that the previous outdoor experiments in a similar environment set the speed of the quadrotor to 0.8 m/s [27]. To compare, the speed of the quadrotor in our real-world experiment was relatively high. From the results, it can be concluded that the proposed path planning method is sufficiently applicable, even in quite complex real-world environments.

6 | CONCLUSIONS

In this study, a practical path planning method combining the A* search algorithm and MSTG was proposed. The A* search algorithm was used to determine a set of waypoints that satisfy geometric constraints to avoid collisions with surrounding obstacles. The MSTG was adopted to generate a smooth trajectory connecting two waypoints, allowing a quadrotor to follow the route efficiently and accurately by using the quadrotor dynamics. To combine the A* search algorithm and MSTG effectively, we made additional efforts to select the essential waypoints (waypoints necessary to generate smooth trajectories), assign time for each segment, and modify the previously generated trajectory. To verify the performance of the proposed path planning method, a fully autonomous quadrotor testbed was configured with a PCL-based environmental perception and PX4 firmware-based flight controller, respectively. As a result, simulation experiments verified that, even in environments with various obstacle densities, a new trajectory could be generated without hitting obstacles in real time (less than 5 Hz), and the generated trajectory was dynamically feasible for a quadrotor to follow. Further, the outdoor experiment confirmed that the quadrotor could fly a total

of 84.23 m in a dense forest over uneven terrain at a maximum speed of 3.22 m/s.

This study can be expanded in the following directions in the future. First, this study uses an inflate map to ensure a safe distance between the quadrotor and the surrounding obstacles. Not only the areas inside of obstacles but also the adjacent areas are considered areas where a quadrotor cannot fly. Therefore, to operate in more complex environments, we need other methods to consider the dimension of the quadrotor. Second, although the MSTG can minimize the snap, it cannot limit the angular velocity within a specific range. If the proposed method is applied for particular problems that require angular velocity reduction, such as localization using vision sensors, we should consider how inequality constraints simultaneously could be integrated into the optimization problem for generating a minimum snap trajectory.

ACKNOWLEDGMENT

Electronics and Telecommunications Research Institute (ETRI) grant funded by the Korean government (2017-0-00067, Development of ICT Core Technologies for Safe Unmanned Vehicles).

CONFLICT OF INTEREST

The authors declare no conflict of interest.

ORCID

Youkyung Hong  <https://orcid.org/0000-0001-5478-9164>

Suseong Kim  <https://orcid.org/0000-0003-0833-8631>

REFERENCES

1. R. Allen and M. Pavone, *A real-time framework for kinodynamic planning with application to quadrotor obstacle avoidance*, in Proc. AIAA Guid. Navig. Control Conf. (San Diego, CA, USA), Jan. 2016.
2. S. Liu et al., *Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-d complex environments*, IEEE Robot. Autom. Lett. **2** (2017), no. 3, 1688–1695.
3. H. Oleynikova et al., *Safe local exploration for replanning in cluttered unknown environments for microaerial vehicles*, IEEE Robot. Autom. Lett. **3** (2018), no. 3, 1474–1481.
4. J. Zhang et al., *Falco: Fast likelihood-based collision avoidance with extension to human-guided navigation*, J. Field Robot. **37** (2020), no. 8, 1300–1313.
5. S. Liu et al., *Search-based motion planning for quadrotors using linear quadratic minimum time control*, IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), **2017** (2017), 2872–2879.
6. J. Huh et al., *Mobile robot exploration in indoor environment using topological structure with invisible barcodes*, ETRI J. **29** (2007), no. 2, 189–200.
7. B. Park, J. Choi, and W. K. Chung, *Incremental hierarchical roadmap construction for efficient path planning*, ETRI J. **40** (2018), no. 4, 458–470.
8. S. M. LaValle, *Rapidly-exploring random trees: A new tool for path planning*, Iowa State University, Ames, IA, USA, Tech. Rep. TR 98-11, 1998.
9. W.-K. Lee, I.-S. Kim, and Y.-D. Hong, *Approach toward foot-step planning considering the walking period: Optimization-based fast footstep planning for humanoid robots*, ETRI J. **40** (2018), no. 4, 471–482.
10. S. Noh et al., *Co-pilot agent for vehicle/driver cooperative and autonomous driving*, ETRI J. **37** (2015), no. 5, 1032–1043.
11. S. Liu et al., *Search-based motion planning for aggressive flight in se (3)*, IEEE Robot. Autom. Lett. **3** (2018), no. 3, 2439–2446.
12. B. Zhou et al., *Robust and efficient quadrotor trajectory generation for fast autonomous flight*, IEEE Robot. Autom. Lett. **4** (2019), no. 4, 3529–3536.
13. D. Mellinger and V. Kumar, *Minimum snap trajectory generation and control for quadrotors*, in Proc. IEEE Int. Conf. Robot. Autom. (Shanghai, China), May 2011, pp. 2520–2525.
14. C. Richter, A. Bry, and N. Roy, *Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments*, in Robotics Research, vol. 114, Springer, Cham, Switzerland, 2016, pp. 649–666.
15. S. Grzonka, G. Grisetti, and W. Burgard, *A fully autonomous indoor quadrotor*, IEEE Trans. Robot. **28** (2011), no. 1, 90–100.
16. D. Mellinger, A. Kushleyev, and V. Kumar, *Mixed-integer quadratic program trajectory generation for heterogeneous quadrotor teams*, in Proc. IEEE Int. Conf. Robot. Autom. (Saint Paul, MN, USA), 2012, pp. 477–483.
17. S. Tang and V. Kumar, *Mixed integer quadratic program trajectory generation for a quadrotor with a cable-suspended payload*, in Proc. IEEE Int. Conf. Robot. Autom. (Seattle, WA, USA), May 2015, pp. 2216–2222.
18. J. Chen, T. Liu, and S. Shen, *Online generation of collision-free trajectories for quadrotor flight in unknown cluttered environments*, in Proc. IEEE Int. Conf. Robot. Autom. (Stockholm, Sweden), May 2016, pp. 1476–1483.
19. H. Oleynikova et al., *Continuous-time trajectory optimization for online uav replanning*, IEEE/RSJ Int. Conf. Intel. Robot. Syst. (IROS), (Daejeon, South Korea), Oct. 2016, pp. 5332–5339.
20. D. Wenchao et al., *Trajectory replanning for quadrotors using kinodynamic search and elastic optimization*, in Proc. IEEE Int. Conf. Robot. Autom. (Brisbane, Australia), May 2018, pp. 7595–7602.
21. F. Gao, Y. Lin, and S. Shen, *Gradient-based online safe trajectory generation for quadrotor flight in complex environments*, in Proc. IEEE/RSJ Int. Conf. Intell. Robot. Syst. (IROS), (Vancouver, Canada), Sept. 2017, pp. 3681–3688.
22. <https://github.com/PointCloudLibrary/pcl/wiki>
23. GitHub Repository, <https://github.com/yrlu/quadrotor>
24. <https://dev.px4.io/v1.9.0/en/simulation/>
25. P. Martin and E. Salaün, *The true role of accelerometer feedback in quadrotor control*, in Proc. IEEE Int. Conf. Robot. Autom. (Anchorage, AK, USA), May 2010, pp. 1623–1629.
26. <http://wiki.ros.org/mavros>
27. F. Gao et al., *Flying on point clouds: Online trajectory generation and autonomous navigation for quadrotors in cluttered environments*, J. Field Robot. **36** (2019), no. 4, 710–733.

AUTHOR BIOGRAPHIES



Youkyung Hong received the BS degree in mechanical engineering from POSTECH, Pohang, South Korea, in 2010, and the MS and PhD degrees in mechanical and aerospace engineering from Seoul National University, Seoul,

South Korea, in 2012 and 2018, respectively. She is currently a researcher at Electronics and Telecommunications Research Institute (ETRI). Her research interests lie in optimization and its application to unmanned vehicle systems.



Suseong Kim received the BS degree in mechanical engineering from Yonsei University, Seoul, South Korea, in 2010, and the PhD degree in mechanical and aerospace engineering from Seoul National University, Seoul, South Korea, in

2017. From 2017 to 2018, he was a Postdoctoral Researcher in Robotics and Perception Group at the University of Zurich and ETH Zurich, Switzerland. He is currently a senior researcher in Electronics and Telecommunications Research Institute (ETRI). His research interests include intelligent control and motion planning of autonomous aerial vehicles.



Yookyung Kim received the BE degree in electrical engineering from the Korea Advanced Institute of Science and Technology (KAIST), Korea, the MS in information and communications engineering from the Gwangju Institute of Science and

Technology (GIST), Korea, and the PhD in electrical

engineering from the University of Arizona, USA, in 2005, 2007, and 2012, respectively. She joined Samsung Electronics, Korea, in 2012, where she was with Samsung Advanced Institute of Technology (SAIT). Since 2014, she has been with the Electronics and Telecommunications Research Institute (ETRI), a national research institute in Korea. Her research interests include compressed sensing, signal modeling, and image processing.



Jihun Cha received the BS degree in computer science from Myongji University, Yongin, South Korea, in 1993, and the MS and PhD degrees in computer science from Florida Institute of Technology, Melbourne, USA, in 1996 and 2002, respectively.

He joined the Electronics and Telecommunications Research Institute (ETRI) in 2003 and currently serves as a directing manager on the autonomous flight research team. His research interests include autonomous flight and counter-UAS.

How to cite this article: Y. Hong, S. Kim, Y. Kim, and J. Cha, *Quadrotor path planning using A* search algorithm and minimum snap trajectory generation*, ETRI Journal **43** (2021), 1013–1023. <https://doi.org/10.4218/etrij.2020-0085>